

Building a Bare-Metal RTOS in Rust for the Raspberry Pi 5

From first UART byte to EL0 user tasks, security hardening, and safety certification — a deep dive into tiny-os, a real-time operating system built across 14 progressive phases.

0x800000	.text.boot	RO+X
	.text	RO+X
	.rodata	RO+NX
0x2000000	.user.text	EL0 X
__data_start	.data	RW+NX
	.bss	RW+NX
	.stack	RW+NX

Scott Davis • September 2026 • Raspberry Pi 5 · BCM2712 · Cortex-A76

tiny-os is a bare-metal real-time operating system written entirely in Rust, targeting the Raspberry Pi 5 and its BCM2712 SoC. It implements preemptive multitasking, SMP across four Cortex-A76 cores, user-space isolation via ARM exception levels, a full network stack, filesystem, cryptographic primitives, and safety certification provisions — all without a single line of C.

14

PHASES

18

HAL TRAITS

21

SYSCALLS

4

SMP CORES

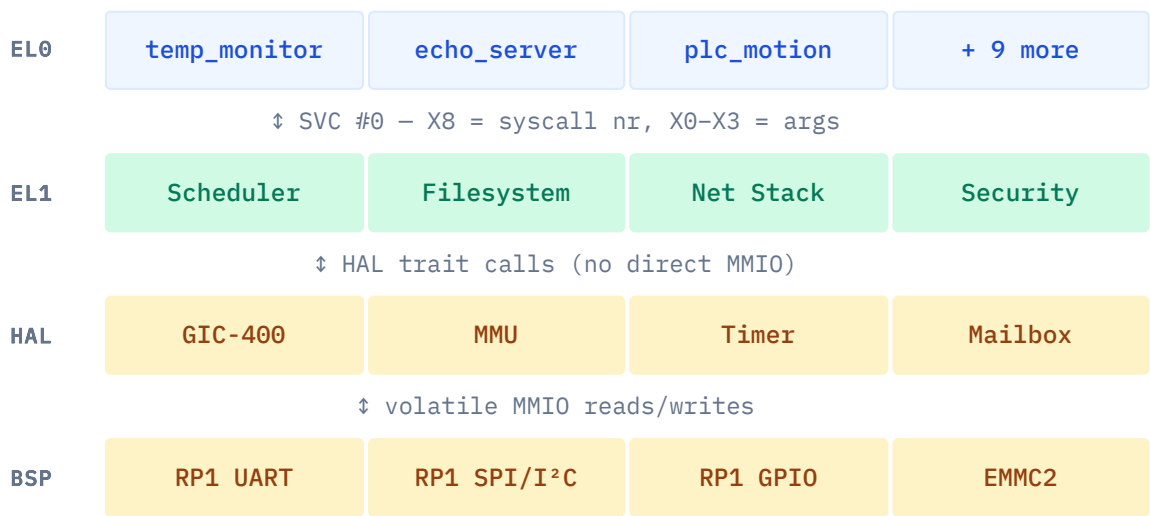
51

HOST TESTS

This article walks through the architecture and key design decisions behind tiny-os — how it boots, manages memory, schedules tasks, isolates user-space code, and hardens itself against both software faults and deliberate attacks. It's written for systems programmers and embedded engineers curious about what a modern, Rust-native RTOS looks like on real ARM hardware.

Architecture: Traits All the Way Down

The foundational rule of tiny-os is simple: **if it touches a hardware register, it goes in `arch/` or `bsp/`, never in `kernel/`**. The kernel talks to hardware exclusively through Rust traits defined in the `arch` crate. This isn't an aspirational design goal — it's how the codebase actually works across 18 HAL traits.



Privilege levels and abstraction layers. The kernel never performs direct MMIO — all hardware access flows through HAL trait implementations.

The workspace is split into three crates. The `arch` crate defines hardware abstraction traits and contains AArch64-specific implementations: boot code, exception vectors, GIC-400 driver, ARM Generic Timer, MMU with per-task page tables, SMP spin-table wakeup,

VideoCore mailbox, and EMMC2 SD controller. The `bsp` crate holds board-specific drivers — RP1 southbridge peripherals for the Pi 5, PL011 UART for QEMU — selected at compile time via Cargo features. The `kernel` crate contains everything hardware-agnostic: scheduler, sync primitives, filesystem, network stack, cryptography, audit log, and user-space management.

This clean separation pays off in the build matrix. `tiny-os` supports six build configurations — three boards (QEMU, Pi 5 standard, Pi 5 safety-critical) times two feature sets (standard and dynamic-load) — and all six compile from the same kernel source.

The HAL trait surface

Each trait defines a minimal, portable contract. Porting `tiny-os` to new hardware means writing trait implementations, not modifying the kernel.

TRAIT	MODULE	PURPOSE
UartDriver	<code>arch::uart</code>	Serial console I/O
InterruptController	<code>arch::irq</code>	GIC / NVIC abstraction
Timer	<code>arch::timer</code>	Periodic tick, monotonic clock
PageAllocator	<code>arch::mm</code>	Physical page frame management
Context	<code>arch::context</code>	Task context save/restore/switch
SmpBoot	<code>arch::smp</code>	Multi-core startup (spin-table)
NetDevice	<code>arch::net</code>	Zero-copy packet TX/RX
UserContext	<code>arch::user</code>	ELO task creation and isolation
SpiDevice	<code>arch::spi</code>	SPI bus configure/transfer
GpioController	<code>arch::gpio</code>	Pin mode, read, write
CryptoEngine	<code>arch::crypto_engine</code>	AES + SHA-256 (hw accelerated)
RtcDevice	<code>arch::rtc</code>	Real-time clock and alarms

The full list includes 18 traits spanning serial, interrupts, timers, memory, DMA, block devices, networking, SPI, I²C, GPIO, PWM, UART ports, RTC, USB, and cryptographic

acceleration. Each one follows the same pattern: a Rust trait in `arch/src/`, an AArch64 implementation in `arch/src/aarch64/`, and optional BSP-level implementations in `bsp/`.

Starting from Nothing

When the Pi 5's firmware loads `kernel8.img` and jumps to `0x800000`, the CPU is in EL2 (hypervisor mode) with the MMU off, caches off, and no stack. The boot sequence in `boot.S` handles the transition to a working kernel environment:

1. **Park secondary cores.** Only core 0 proceeds; cores 1–3 enter a WFE spin loop watching release addresses in `.data`, to be woken later by the SMP subsystem.
2. **Drop privilege.** Configure EL2 registers (HCR_EL2 for AArch64, SCTLR_EL1 defaults), then `eret` to EL1. On QEMU, which boots at EL3, we drop through EL3→EL2→EL1.
3. **Zero .bss, set stack pointer.** The linker script exports `__bss_start` and `__bss_end`; the boot code zeroes that range. The stack pointer is set to the top of the `.stack` section (512 KB for the primary core).
4. **Preserve the DTB pointer.** The firmware passes the device tree blob address in x0. `Boot.S` stashes it in x19 (callee-saved), which `kmain` later stores in a global for memory discovery.
5. **Branch to `kmain()`.** From here, it's Rust. The kernel initializes subsystems in dependency order: UART, interrupts, MMU, heap, scheduler, SMP, storage, filesystem, network, user tasks, shell.

The entire boot path through assembly is about 120 lines. Everything after `kmain()` is safe Rust (with `unsafe` blocks for hardware register access, each annotated with a `// SAFETY: comment`).

Memory Management Without a Kernel to Lean On

Memory management starts with a minimal FDT parser that reads the `/memory` node from the firmware-provided device tree to discover available RAM. If parsing fails, BSP-provided defaults are used (1 GB for QEMU, 4 GB for Pi 5).

Physical memory

The page frame allocator is a bitmap — one bit per 4 KB page, supporting up to 4 GB (1 million pages). It's simple, deterministic, and has bounded worst-case allocation time, which matters for an RTOS. A contiguous `alloc_pages` variant exists for DMA buffers that require physically contiguous memory.

MMU setup

The MMU uses AArch64's 4 KB granule with 2 MB block descriptors for the identity mapping. Three MAIR indices define the memory types: Device-nGnRnE for MMIO, Normal Write-Back for RAM, and Normal Non-Cacheable for DMA buffers. A strict **W^X policy** is enforced from boot:

- **Code** (`.text` , `.rodata`): Read-only + Execute — `AP_R0_EL1` , no `PXN`
- **Data** (`.data` , `.bss` , heap, stacks): Read-Write + No Execute — `AP_RW_EL1` , `PXN` set
- **MMIO** (peripherals, RP1 window): Read-Write + No Execute, Device memory type

The linker script aligns `__data_start` to a 2 MB boundary to ensure the W^X split maps cleanly onto block descriptors — no page needs both write and execute permissions.

Per-task page tables

User tasks each get their own TTBR0 page table, cloned from the kernel's identity mapping. The key addition is L3 page tables (4 KB granularity) for user stack regions, including a guard page with no access permissions to catch stack overflow. Each user task receives an 8-bit ASID (Address Space Identifier), so TLB entries from different tasks don't collide. The scheduler swaps TTBR0 and issues a TLBI on context switch when the incoming task has a different address space.

Static pools size the system: up to 16 user tasks, 20 L2 table entries, and 32 L3 tables. These are allocated as static arrays initialized at compile time using const expressions — no dynamic allocation required.

Scheduling: 256 Priorities, Four Cores

The scheduler is a 256-level fixed-priority preemptive design with $O(1)$ dispatch. The ready queue uses four `u64` bitmaps (covering priorities 0–255), and dispatch is a single `trailing_zeros` operation on the first non-zero word. Equal-priority tasks are scheduled round-robin via per-level FIFO linked-list queues.

RUST

kernel/src/sched.rs

```
// O(1) dispatch: find the highest-priority non-empty queue
let word = 0;
while word < 4 {
    if self.bitmap[word] != 0 {
        let bit = self.bitmap[word].trailing_zeros() as usize;
        let prio = word * 64 + bit;
        return self.dequeue(prio);
    }
    word += 1;
}
```

Tasks have five states — Ready, Running, Blocked, Suspended, and Dormant — and four criticality levels: SafetyCritical, MissionCritical, Standard, and BestEffort. The criticality level governs behavior during mode switches: when the system enters a degraded state, lower-criticality tasks are suspended to free resources for safety-critical work.

SMP

All four Cortex-A76 cores share a single global run queue, protected by a ticket spinlock with IRQ save/restore. Each core tracks its own current task and has its own idle task (priority 255, WFE loop). When a core adds a task to the ready queue and detects that

another core is idle, it fires a GIC SGI #0 (Software Generated Interrupt) to wake that core for rescheduling.

The scheduler lock is held across context switch and released by the resumed task — or by `task_trampoline` for newly created tasks. This protocol prevents a window where the lock is released but the new task's stack frame hasn't been fully set up.

Synchronization primitives

Built on the scheduler's blocking infrastructure: mutexes with Priority Inheritance and Priority Ceiling protocols, counting/binary semaphores, 32-bit event flags with Any/All wait modes, and const-generic message queues. All primitives support timeouts and non-blocking try variants. The WaitQueue maintains waiters in priority order with lazy stale-entry cleanup.

The User-Space Boundary

tiny-os enforces hardware-backed isolation between user applications and the kernel using ARM exception levels. User tasks run at EL0 (unprivileged); the kernel runs at EL1. This is the same privilege model used by Linux, but in a much simpler form.

Task creation

A user task is created via `task_create_user`, which allocates a kernel stack (for syscall handling), a user stack (with L3 page table mapping and guard page), and a TTBR0 page table. The task's entry point lives in the `.user.text` linker section at `0x200000`, identity-mapped with EL0-accessible permissions.

The task starts on a special trampoline (`task_trampoline_user` in assembly) that releases the scheduler lock, sets `SPSR_EL1` to `0x0` (EL0t mode — EL0 with `SP_EL0`), loads the entry point into `ELR_EL1`, sets `SP_EL0` to the user stack top, and executes `eret`. From that point, the task runs at EL0 with no access to kernel memory or hardware registers.

Syscall interface

User tasks request kernel services via `SVC #0`. The calling convention places the syscall number in X8 and arguments in X0–X3. The return value comes back in X0.

0 <code>SYS_YIELD</code>	1 <code>SYS_DELAY</code>	2 <code>SYS_WRITE</code>
3 <code>SYS_TASK_ID</code>	4 <code>SYS_UPTIME</code>	5 <code>SYS_EXIT</code>
6 <code>SYS_TEMPERATURE</code>	10 <code>SYS_FS</code>	11 <code>SYS_NET</code>
12 <code>SYS_SPI</code>	13 <code>SYS_I2C</code>	14 <code>SYS_GPIO</code>
15 <code>SYS_UART</code>	16 <code>SYS_PWM</code>	17 <code>SYS_RTC</code>
18 <code>SYS_DMA</code>	19 <code>SYS_USB</code>	20 <code>SYS_CRYPTO</code>
21 <code>SYS_POWER</code>		

Subsystem syscalls (10–21) are multiplexed: X8 selects the subsystem, X0 selects the operation within it, and X1–X3 carry arguments. This keeps the syscall number space compact while exposing rich functionality — the filesystem alone supports open, read, write, close, create, and directory enumeration through a single syscall number.

Capability enforcement

Every task carries a 32-bit capability bitmask in its TCB. Kernel tasks get `CAP_ALL`; user tasks get `CAP_USER_DEFAULT`, which grants yield, delay, write, task ID, uptime, exit, temperature, filesystem, and networking — but **excludes** SPI, I²C, GPIO, UART, PWM, RTC, DMA, USB, crypto, and power management. A user task requesting a denied syscall receives `E_PERM` (`u64::MAX - 8`) and the attempt is logged to the audit trail.

This means all 12 example applications compile against the same syscall interface, but those targeting restricted peripherals (like the LED blinker using GPIO, or the crypto signer using `SYS_CRYPTO`) detect the denial at runtime and fall back gracefully — blinking in simulation mode, or using a software XOR checksum instead of hardware SHA-256.

Writing a user-space application

User applications are pure `#[no_std]` Rust functions with every item annotated `#[link_section = ".user.text"]`. Since the code runs at EL0 with no access to kernel memory, all string constants must also live in `.user.text`. Buffers use `MaybeUninit` and are accessed exclusively through `read_volatile` / `write_volatile` to prevent the compiler from generating `memcpy` calls (which would reference `libc` symbols unavailable in bare-metal).

RUST

examples/temp_monitor/main.rs – syscall wrapper

```
#[link_section = ".user.text"]
#[inline(always)]
fn syscall(nr: u64, a0: u64, a1: u64) -> u64 {
    let ret: u64;
    unsafe {
        asm!(
            "svc #0",
            in("x8") nr,
            inout("x0") a0 => ret,
            in("x1") a1,
            lateout("x2") _,
            lateout("x3") _,
            );
    }
    ret
}
```

Security: Defense in Depth

tiny-os implements security hardening across five layers, targeting the attack patterns that compromise industrial control systems.

Network firewall

An allowlist-based packet filter with up to 16 rules operates on a default-deny policy. Every inbound packet is checked against source IP/mask, destination port, and protocol before

reaching the network stack. Rejected packets increment atomic drop counters and generate audit entries. The firewall sits in the IPv4 receive path, before protocol demultiplexing — there's no way for a packet to bypass it.

Cryptographic primitives

SHA-256 (FIPS 180-4) and HMAC-SHA256 (RFC 2104) are implemented in software with constant-time comparison for verification. CRC32 uses a precomputed 256-entry lookup table. On Pi 5 hardware, ARMv8 Crypto Extensions provide hardware-accelerated AES (ECB, CBC, CTR modes) using AESE/AESD/AESMC/AESIMC instructions, detected at runtime via `ID_AA64ISAR0_EL1`.

Runtime code integrity

At boot, a CRC32 is computed over the entire `.text` section (from `_start` to `__data_start`). The health monitor periodically re-computes this checksum and compares it against the boot value. Any mismatch triggers `os_hook_health_check_failed("code_integrity")` and logs an `IntegrityFail` audit event.

Pointer Authentication (ARMv8.3 PAC)

On Cortex-A76 (Pi 5), tiny-os enables Pointer Authentication for return addresses. The APIA key is initialized from encoded system registers, and SCTLR_EL1's EnIA bit enables PACIASP/AUTIASP instruction authentication. This makes ROP (Return-Oriented Programming) attacks significantly harder — a corrupted return address will fail authentication and trap. PAC is detected at runtime and cfg-gated to `bsp-rpi5` only, since QEMU's Cortex-A72 emulation doesn't support ARMv8.3.

Secure memory wiping

When a task terminates, its entire stack and sensitive TCB fields (saved SP, capabilities, syscall counters, CPU time) are zeroed using `write_volatile` to prevent compiler elision. This ensures that when the memory is recycled for a new task, no data from the previous tenant is observable.

Syscall rate limiting

Each task has a sliding window rate limiter: a counter and window start timestamp in the TCB. The default allows 1,000 syscalls per second. When exceeded, further syscalls return `E_RATE_LIMIT ( u64::MAX - 9 )` until the window slides past. This prevents a compromised or runaway user task from monopolizing kernel time with syscall storms. Violations are logged to the audit trail.

Audit log

A 64-entry ring buffer records 11 event types: Boot, Shutdown, AuthOk, AuthFail, FirewallDrop, CapabilityDenied, IntegrityOk, IntegrityFail, TaskCreated, TaskTerminated, and RateLimited. Each entry records the tick, core ID, task ID, event type, and a 40-byte detail string. The log can be persisted to a FAT32 file via the shell's `audit persist` command.

Safety Certification Provisions

The specification targets IEC 61508 SIL-2, ISO 26262 ASIL-B, and DO-178C DAL-C. While tiny-os hasn't undergone formal certification, the architecture includes the evidence and mechanisms these standards require.

The `safety-critical Cargo feature` is a compile-time switch that changes the system's behavior: the heap is disabled (pool-only allocation), budget monitoring is mandatory, the watchdog must be explicitly kicked, and shell authentication is enforced. This models the difference between a development build and a production deployment in a certified system.

WCET measurement. A harness built on the PMU cycle counter (`PMCCNTR_EL0`) tracks min/max/average execution times for instrumented kernel paths. The results feed into schedulability analysis — RMA utilization checks and iterative Response-Time Analysis with Priority Inheritance Protocol blocking terms.

Health monitoring. A dedicated task (priority 1) runs eight checks every five seconds: stack watermarks, CPU utilization, watchdog status, ready queue integrity, mutex ownership consistency, tick counter monotonicity, pool allocator accounting, and code integrity verification. Failures invoke configurable hook functions that can log, alert, or trigger a controlled shutdown.

Mixed-criticality partitioning. The `os_criticality_switch` API suspends all tasks below a given criticality level, freeing resources for safety-critical work.

`os_criticality_restore` resumes them. Combined with per-task execution-time budgets and automatic suspension on overrun, this provides temporal partitioning without hardware virtualization extensions.

Traceability. A CSV matrix maps 86 requirements (REQ-CFG-*, REQ-POOL-*, REQ-SAFE-*, REQ-SEC-*, etc.) bidirectionally between specification sections, source files, and test cases. This is the primary artifact auditors review during certification.

The 12 Example Applications

Each example runs as an independent EL0 task, communicating with the kernel exclusively through syscalls. They range from simple sensor monitors to industrial control simulations, demonstrating the full breadth of the syscall interface and graceful degradation when capabilities are denied.

temp_monitor

SoC temperature via VideoCore mailbox. Tracks min/max/avg, reports every 5 seconds.

TEMPERATURE WRITE DELAY

sensor_gateway

Industrial sensor data collection via SPI/I²C/GPIO, SD card logging, UDP telemetry.

SPI I2C GPIO FS NET

system_dashboard

Periodic status reports: uptime, task ID, temperature. Lightweight system health view.

UPTIME TASK_ID TEMPERATURE

data_logger

Creates files on the FAT32 filesystem, writes periodic sensor readings.

FS TEMPERATURE UPTIME

echo_server

UDP echo server on port 7777. Demonstrates socket lifecycle from user space.

NET WRITE

led_blinker

GPIO/PWM blink patterns (steady, heartbeat, SOS). Falls back to simulated output.

GPIO PWM DELAY

rate_limit_demo

Deliberately triggers syscall rate limiting, observes E_RATE_LIMIT, then recovers.

YIELD UPTIME TASK_ID

plc_motion

5-state PLC motion controller with trapezoidal velocity profile, 10 ms scan cycle.

DELAY WRITE UPTIME

machine_vision

32×32 frame inspection pipeline: acquire, threshold, label, measure, classify pass/fail.

DELAY WRITE UPTIME

crypto_signer

Signs log records with SHA-256 (or XOR fallback), writes to /SIGNED.TXT on disk.

CRYPTO FS TEMPERATURE

power_monitor

DVFS frequency/voltage monitoring via SYS_POWER, thermal warnings above 70°C.

POWER TEMPERATURE

rtc_clock

Real-time clock display with 60-second recurring alarms and uptime-only fallback.

RTC UPTIME DELAY

Applications that use restricted syscalls (SPI, GPIO, CRYPTO, POWER, RTC) demonstrate the capability system in action. When running under `CAP_USER_DEFAULT`, these calls return `E_PERM`, and the application detects this on its first probe and switches to a degraded mode — continuing to run and produce useful output rather than crashing. This pattern mirrors how real embedded systems handle optional hardware features.

Network Stack

The network stack implements Ethernet frame parsing, a 16-entry ARP cache, IPv4 with internet checksum, ICMP echo request/reply, UDP with a port table, and a minimal TCP state machine (client-only SYN/ACK/FIN, four connections, no retransmission or congestion

control). It's enough to respond to pings, serve UDP echo traffic, and make outbound TCP connections.

For QEMU testing, a loopback `NetDevice` handles packets entirely in software — swapping source/destination addresses and converting ICMP echo requests into replies. On real hardware, the RP1 Gigabit Ethernet MAC (Synopsys GMAC) would implement the same trait. The BSD-style socket API (`socket` , `bind` , `connect` , `sendto` , `recvfrom` , `close`) is exposed to user space through `SYS_NET` .

The zero-copy buffer pool (1,024 × 1,536-byte buffers in a 2 MB non-cacheable memory region) backs all packet I/O. Buffers use atomic reference counting and a spinlock-protected free list, so packets can be handed between the network task and user-space syscall handlers without copying.

Storage and Filesystem

The storage subsystem routes between two block devices: a real SDHCI/EMMC2 driver on Pi 5 hardware (supporting SDR104 UHS-I at 208 MHz with ADMA2 DMA transfers and graceful fallback to 25 MHz PIO mode) and a 256 KB RAM disk formatted as FAT32 on QEMU. Both implement the `BlockDevice` trait and sit behind an LRU write-back block cache (32 lines of 512 bytes each).

The filesystem is FAT32 with a VFS abstraction layer: BPB parsing, FAT chain traversal, directory parsing with long filename support, case-insensitive name matching, and file create with 8.3 short name generation. A 16-entry file descriptor table manages open files. The `SYS_FS` syscall exposes create, read, write, close, and directory enumeration to user space.

Testing Without an OS

Testing a bare-metal OS presents a bootstrapping problem: you can't run `cargo test` when your default target is `aarch64-unknown-none` with no standard library. `tiny-os` solves this with a two-tier approach.

Host-side unit tests (51 tests) re-implement pure-logic algorithms — IPv4 checksums, Ethernet frame parsing, MBR partition tables, SHA-256/HMAC-SHA256, CRC32, and RTC datetime conversions — in a separate `tests/host` crate that builds natively with `--target x86_64-pc-windows-msvc`. These test the algorithms that back network processing, cryptography, and storage without needing an ARM emulator.

QEMU integration tests (15 checks) boot the full kernel on QEMU `-M raspi4b`, capture serial output with a 15-second timeout, and verify expected patterns: kernel banner, UART init, MMU enable, timer accuracy, scheduler start, all four SMP cores online, network loopback, filesystem mount, user task running at EL0, shell prompt, and no kernel panic. This is a full end-to-end verification of the boot sequence and subsystem initialization.

The 14-Phase Build

`tiny-os` was built incrementally, each phase adding a major subsystem and its dependencies. Every phase leaves the system bootable and testable — there's no phase that requires the next one to produce a working kernel.

01	Bootstrap & UART	✓	02	Interrupts & Timer	✓
03	Memory Management	✓	04	Multitasking	✓
05	Synchronization	✓	06	Drivers & Logging	✓
07	SMP (4 cores)	✓	08	Storage & DMA	✓
09	Filesystem (FAT32)	✓	10	Networking & User Mode	✓
11	Safety Certification	✓	12	Extended Peripherals	✓

The progression is deliberate. You can't do multitasking (Phase 4) without memory management (Phase 3), which needs the MMU, which needs the page allocator. You can't do SMP (Phase 7) without synchronization primitives (Phase 5). User mode (Phase 10) builds on virtually everything before it. Security (Phase 13) and safety (Phase 11) are layered on top of a working system — they harden it rather than define it.

What's Next

The stretch goals include VideoCore VII GPU access (framebuffer and compute shaders), HDMI output, MIPI CSI camera input, Wi-Fi (CYW43455 over SDIO), Bluetooth 5.0/BLE, NVMe storage via PCIe, a POSIX-compatible process model with `fork / exec`, async cooperative tasks alongside preemptive ones, and ports to Cortex-M targets (RP2040, STM32).

The project is open source under the MIT license. The entire codebase — 14 phases of implementation, 18 HAL traits, 21 syscalls, 12 example applications, 51 host tests, and 15 QEMU integration checks — was built in Rust with Claude Code assistance.

For CSU Sacramento students: this project was created to show what's possible with AI-assisted development tools and what a well-structured GitHub project looks like. If you're looking for an internship or about to graduate, put your projects on GitHub. Employers want to see what you can build — and a public repo is the easiest way to show them.